

SW Engineering CSC 648-848 Spring 2025

Project Name: GatorTrade

Team 08

Team Lead: Jonathan Carbajal - Jcarbajal@sfsu.edu

Backend Lead: Christian Kurt Balais - cbalais@sfsu.edu

Frontend Lead: Tung-Ying Lee - nlee@sfsu.edu

Frontend Lead 2: Chia-Tung Liu - cliu26@sfsu.edu

Github Master: Luis Moreno - lmoreno10@sfsu.edu

Milestone 4

Date: 05/20/2025

History Table:

Version:	Date Submitted:	Date Revised:
M4	5/20/2025	5/20/2025

Table of Contents

I. Product Summary.....	1
II. Usability Test Plan.....	1
III. QA Test Plan and Testing.....	3
IV. Peer Code Review:.....	6
V. Self-Check for Security.....	9
VI. Self-Check for Original Non-functional specs.....	10
VII. Use of GenAI.....	11

I. Product Summary:

GatorTrade is a safe, student-focused online marketplace. Understanding the drawbacks and dangers of popular websites like Facebook Marketplace, Craigslist, OfferUp, and eBay, our team developed GatorTrade to specifically address the difficulties SFSU students encounter when purchasing and selling daily necessities like furniture, electronics, and textbooks. GatorTrade's closed-network environment, which only allows individuals with validated SFSU email addresses to join, is what makes it unique. This considerably lowers the possibility of scams, spam, or anonymous malicious users by guaranteeing that all interactions occur inside a recognized and trusted academic community. GatorTrade was created with student safety and convenience as its top priorities, in contrast to larger platforms that place more emphasis on reach and ad revenue.

Major Committed Functions: P1

- **Browsing Items:**
Unregistered users can browse all items, including categories, prices, images, and descriptions.
- **Searching & Filtering:**
Users can search by keyword, filter by category, and set a price range to narrow down listings.
- **Viewing Item Details:**
Everyone can view full item information — including images, full descriptions, prices, and posting time.
- **Student Registration:**
New users can sign up using their valid university (SFSU) email address.
- **Login Access:**
Registered users can log in to access posting and messaging features.
- **Posting Items for Sale:**
Logged-in users can list items by submitting details and images, which go through admin approval before being published.
- **Managing Listings:**
Sellers can delete their listings or mark items as sold when no longer available.
- **In-Site Messaging:**
Buyers can contact sellers through a private in-platform messaging system.
- **Seller Inbox & Notifications:**
Sellers are notified via automated email (sent to their SFSU inbox) when a buyer sends them a message for the first time.
- **Profile Management:**
Users can update some of their personal information

- **Safe Meetup Locations:**

The platform suggests secure on-campus meetup spots that users can choose for safe exchanges in messages. Choosing a location sends a prewritten message.

URL: <http://18.117.162.109/>

II. Usability Test Plan:

A. Test Objectives

- Evaluate how effectively and efficiently users can search for listings based on category and text input. The goal is to assess the intuitiveness of the UI and how well users understand and complete the task.

B. Test Background and Setup

- System setup: Deployed React + Express app, backend connected to MySQL DB.
- Tester Hardware: Desktop/laptop with a modern browser (Chrome, Firefox).
- Target Users: SFSU students interested in peer-to-peer item exchange.
- URL: <http://<your-ec2-ip>>
- Environment: Home setting, no cameras or monitoring; no training required.

C. Usability Task Description

- You are a student looking to buy a used textbook. Please:
 - a) Go to the website.
 - b) Use the category dropdown to select "Books".
 - c) In the search bar, type a relevant keyword (e.g., "Calculus").
 - d) Click the "Search" button.
 - e) View the results, and try to find the most relevant listing.
 - f) Tell us how confident you are that you found what you sought.

D. Evaluation of Effectiveness

- Effectiveness will be measured by:
 - a) Whether the correct results are returned based on the input
 - b) Whether users were able to complete all tasks
 - c) Observations of any mistakes, confusion, or abandonment

E. Evaluation of Efficiency

- Efficiency will be measured by:
 - a) Time taken from task start to successful completion
 - b) Number of clicks and interactions
 - c) User hesitation or repeated actions

F. Evaluation of User Satisfaction

- Testers will answer these after the task:
 - a) "It was easy to find and use the search function."
→ 1 (Strongly Disagree) to 5 (Strongly Agree)

- b) "The search results were relevant to my input."
→ 1 to 5 scale
- c) "I would use this site to search for items again."
→ 1 to 5 scale

G. GenAI Use:

- Tool: ChatGPT (GPT-4)
- Use: Guide on how to do the test plan, refined Likert questions
- Prompt Example: "What do I need to draft a usability test plan?"
- Utility: High

III. QA Test Plan and Testing:

1. Test Objectives

To verify that the search functionality:

- Accurately returns results based on entered keywords
- Displays correct and relevant school entries
- Handles incorrect or empty inputs gracefully
- Functions properly across major web browsers

2. HW and SW Setup

- **Hardware:** Standard laptop (Intel i9 ,16GB RAM)
- **Operating Systems:** Windows 11
- **Browsers Used for Testing:**
 - Google Chrome v123
 - Safari
- **URL:** <http://localhost:5000/> , <http://18.117.162.109/>

3. Feature to Be Tested

Search Function on Homepage and Search Results Page

The user enters a school name or keyword into the search bar. The system returns matching results based on school name, location, or tags.

4. QA Test Plan Table

Test #	Test Title	Test Description	Test Input	Expected Output	Chrome	Safari
1	Basic Search	Tests keyword match the result	chair	Results showing all chairs from the market	PASS	PASS
2	Invalid/Empty Search	Tests handling of blank or unrelated inputs	asdfgh / blank	Message: “No results found”, for blank it just show all the result on the website	PASS	PASS
3	Case-insensitive Search	Check if uppercase/lowercase query returns same results	BOOK	Same results as book	PASS	PASS
4	Partial Match	Input a partial word to test for autocomplete or partial string search match	ch	Results include items where either the name or description contains “ch”.	PASS	PASS
5	Search Result Click-through	Click on a search result and verify redirection to correct items	Click selling item	Redirects to full item info page	PASS	PASS
6	Search Input Limit	Ensure the input does not accept more than 40 characters	A string longer than 40 characters	An alert shown to user.	PASS	PASS

5. Test Results Summary

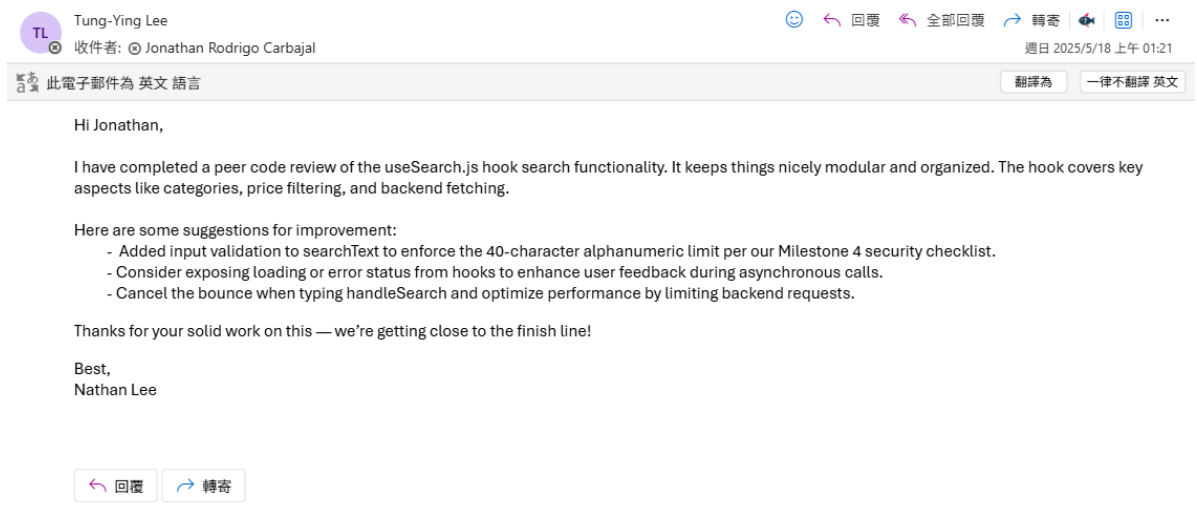
- Testing was executed manually on both Chrome and Firefox.
- All five test cases passed successfully on both browsers.

- No functional bugs or UI rendering issues were observed.

6. GenAI Use Summary

- **Tool Used:** ChatGPT (GPT-4)
- **Usage:** Assisted in drafting the test plan structure, test case ideas, and refining test input/output expectations.
- **Prompts Used:**
 - “Help write a QA test plan table for a school search website feature.”
 - “What are good test cases for a search input form?”
- **Benefit:** Helped ensure comprehensive test coverage, clearly written cases, and professionally formatted table.
- **Utility Ranking: HIGH** – Saved time and improved clarity of documentation.

IV. Peer Code Review:



```

1  import { useState } from 'react';
2
3  export function useSearch(initialCategory = '', initialText = '', initialResults = []) {
4    const [searchCategory, setSearchCategory] = useState(initialCategory);
5    const [searchText, setSearchText] = useState(initialText);
6    const [results, setResults] = useState(initialResults);
7    const [minPrice, setMinPrice] = useState(0);
8    const [maxPrice, setMaxPrice] = useState(1000);
9
10   const handleSearch = async (
11     categoryParam = searchCategory,
12     textParam = searchText,
13     min = minPrice,
14     max = maxPrice
15   ) => {
16     try {
17       const params = new URLSearchParams({
18         category: categoryParam,
19         query: textParam,
20         minPrice: min,
21         maxPrice: max,
22       });
23
24       const response = await fetch(`/items/search?${params.toString()}`);
25       const data = await response.json();
26
27       if (response.ok) {
28         setResults(data);
29       } else {
30         console.error("Search error:", data.error);
31       }
32     } catch (error) {
33       console.error("Fetch error:", error);
34     }
35   };
36
37   return {
38     searchCategory,
39     setSearchCategory,
40     searchText,
41     setSearchText,
42     results,
43     setResults,
44     minPrice,
45     setMinPrice,
46     maxPrice,
47     setMaxPrice,
48     handleSearch,
49   };
50 }

```

File Reviewed: src/hooks/useSearch.js

Functionality: Search hook for managing query input, category filter, price range, and backend fetch

What's Done Well

- Clarity and readability: The code is well-structured and clean. You used consistent formatting and spacing.
- Naming conventions: Function and variable names (such as handleSearch, setSearchText, and minPrice) are self-explanatory, following the camelCase convention.
- Functional cohesion: State management is tightly grouped, making it easy to extend filtering or updating logic.
- Modularity: Wrap your search logic in reusable hooks to make your app scalable and DRY.

Improvement Suggestions

- Input Validation:
 - This hook does not enforce any character length restrictions on searchText. Add the following checks:
if (textParam.length > 40 || /^[^a-zA-Z0-9]/.test(textParam)) return;
- Error feedback:
 - If an API fetch fails or returns an error, the user never sees it.
 - Consider passing an error message or a boolean flag back from handleSearch so your UI can display that information.
- Performance - Debounce:
 - Fast input will send multiple GET requests.
 - It is recommended to wrap handleSearch in a debounce utility when integrating into components to reduce unnecessary API calls.
- Missing loading status:
 - The user has no visual cue while the results are loading.
 - It is recommended to add isLoading to the return status of your hook and toggle it during asynchronous fetching.

GenAI Code Review Summary

Tool: ChatGPT

Prompt Used:

Hi, I'm a senior Computer Science student reviewing React custom hooks for search functionality in my team's marketplace app. Can you help me evaluate the code to see if there are improvements in structure, performance, input validation, and overall reusability? Please highlight suggestions that would be helpful for app expansion or for others to use later.

GenAI Review Results:

- Separate API Logic from State Logic
- "It would be a good idea to move the fetch call into a separate utility or service file. That way, if the API endpoint changes or is reused elsewhere, you won't have to update the hook itself."

Expose Reset Functionality

- "Add a resetFilters() function to clear search fields and filters. This would make it easier to add a reset button in the UI and improve usability."

Transform API Response

- "You could shape the response data before setting it into state. For example, map or filter fields you'll actually need in the UI. This will make the UI rendering cleaner and more maintainable."

Improve Documentation

- “Try adding brief comments for each state variable and return value, especially `handleSearch()`. Even short inline descriptions help other developers (or your future self) understand it faster.”

Scalability Note

- “As your app scales, you might want to include pagination or sorting support. Structuring the hook to handle those now (even as optional params) could save refactoring later.”

GenAI Utility RANK: HIGH

This GenAI review provided a different angle from the peer review by:

- Focusing on code reusability, modularity, and growth readiness
- Suggesting improvements that make the hook more maintainable and scalable
- Offering ideas you can use even if the current version works fine

It complements the peer review by avoiding overlap and showing how the hook could evolve as the project grows or enters production.

V. Self-Check for Security:

Asset To Be Protected	Types of possible/expected Attacks	Consequences of a Security Breach	Strategy to mitigate/protect the asset
Registration	Non-accepted values inserted.	Unauthorized accounts created.	Regex checks if the school email is used, frontend AND backend both check if the inputs are acceptable.
Full Name	Impersonation, phishing	Identity misuse	User review, deletion from database (erase unaccepted accounts).
Passwords	Login Brute-force	Unauthorized access	Passwords are hashed using bcrypt in the backend before saving to the DB.
Messages	Unauthorized access, URL manipulation	Privacy breach, access to private communications	Backend checks both sender & recipient IDs before returning messages. URL tampering fails in the backend.
Listing (Status)	Fake or malicious posts	Scams or misinformation	Admins must approve listings before public display. Status defaults to <i>pending</i> .
Listings (ownership)	Unauthorized UserID reassignment	Data spoofing, impersonation	Only DB admins can reassign userIDs.
Listing (images)	Inappropriate content, non-image uploads	Content policy violations	All images are reviewed before listing approval. The backend limits file type & size.
Session State	Session hijack, improper logout	Unauthorized actions from stale sessions	Sessions expire after 24 hours or on server restart. No persistent login allowed.
Unauthorized Deletion	User deletes someone else's listing	Loss of data, abuse of access	Backend checks listing ownership (userID) before allowing deletes. Only admins can delete listings in workbench.

Additional Nodes:

- **Passwords are encrypted** using bcrypt before being saved in the database.
- Search Bar ONLY accepts 40 alphanumeric characters max
- Registration checks if the email ends with "sfsu.edu".
- Dummy terms of service are in registration, and must be accepted.

VI. Self-Check for Original Non-functional specs:

1. The application shall be developed, tested, and deployed using tools and cloud servers approved by Class CTO and as agreed in M0. **DONE**
2. The application shall be optimized for standard desktop/laptop browsers; e.g., it must render correctly on the two latest versions of two major browsers. **DONE**
3. All or selected application functions shall render well on mobile devices (no native app to be developed). **DONE**
4. Posting of sales information and messaging to sellers shall be limited only to SFSU students. **DONE**
5. Critical data shall be stored in the database on the team's deployment server. **DONE**
6. No more than 50 concurrent users shall be accessing the application at any time. **DONE**
7. The privacy of users shall be protected. **DONE**
8. The language used shall be English (no localization needed). **DONE**
9. The application shall be easy to use and intuitive. **DONE**
10. The application shall follow established architecture patterns. **DONE**
11. The application code and its repository shall be easy to inspect and maintain. **DONE**
12. Google Analytics shall be used. **DONE**
13. No e-mail clients or chat services shall be allowed. Interested users can only message sellers via in-site messaging. One round of messaging (from user to seller) is enough for this application. **DONE**
14. Pay functionality, if any (e.g., paying for goods and services), shall not be implemented nor simulated in UI. **DONE**
15. Site security: Basic best practices shall be applied (as covered in the class) for main data items. **DONE**

16. Media formats shall be standard as used in the market today. **DONE**
17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools. **DONE**
18. The application UI (WWW and mobile) shall prominently display the exact text on all pages: "SFSU Software Engineering Project CSC 648-848, Spring 2025. For Demonstration Only" at the top of the WWW page Nav bar. (It is important to not confuse this with a real application). You have to use this exact text without any editing. **DONE**

VII. Use of GenAI:

Brainstorming Ideas:

ChatGPT was used to help brainstorm and refine final milestone features, including search filtering logic, user session handling, and enhancements to the item listing and header components. It provided suggestions for category design, helped identify potential pain points in filtering implementation, and assisted in defining the logic behind secure login state handling.

Coding Support:

Gen ai was used to help with bugs and error codes in the terminal. It was used a lot to learn react and check code for errors since it was our first time with react. Helped with troubleshooting filtering. Mostly used for guidance and help

Writing and Structure:

ChatGPT was used to clarify error messages, simplify conditional logic, and improve the readability of routes and controller files. Additionally, it was instrumental in helping refine report content and writing the product summary by expanding initial draft content with clearer organization and a more polished tone.

Functional Guidance:

It helped identify a clean approach to integrating price filters without disrupting existing search logic and guided the integration of dynamic header states using session-based login detection. ChatGPT also provided clear direction on how to style user-dependent components using conditional rendering.